

A Supervised ROS 2 Architecture for Autonomous Mapping: Modular Exploration-Management Framework

Roberto Raymundo Gómez Vargas[†] and Rafael Cisneros-Limón^{†*}

Abstract—This paper presents a modular ROS 2 architecture for autonomous mapping in mobile robots operating within unknown and dynamic indoor environments. The proposed system integrates multimodal scan fusion, a static–dynamic obstacle-filtering module, SLAM-based map construction, frontier-based exploration, and Nav2-based navigation, coordinated by a program manager that governs the mission from distributed launch through coordinated termination. Beyond this supervised software system, the architecture is organized around a modular ROS 2 exploration-management framework that extends beyond frontier detection to govern candidate evaluation, navigation-goal supervision, failure recovery, and mission completion. Requiring only an occupancy-grid map, robot-pose information through TF, and a Nav2-compatible navigation interface, the framework may be adopted by compatible ROS 2 mobile-robot stacks through external configuration, without modification of its underlying implementation. The complete system is validated in simulation and on the physical CALL-M mobile manipulator.

Index Terms—Autonomous mapping, exploration management, software architecture, modular framework, ROS 2.

I. INTRODUCTION

Autonomous mapping is a fundamental capability for mobile robots operating in unknown environments. Within ROS 2, mature frameworks such as SLAM Toolbox [1] and Nav2 [2] provide established solutions for mapping, localization, navigation, and motion execution. Nevertheless, a complete autonomous mapping mission requires more than the coexistence of these components: the system must determine where to explore next, supervise navigation goals, recover from failures, avoid repeated pursuit of unproductive regions, assess continued progress, and ultimately bring the mission to completion. Existing exploration methods [3], [4], [5], [6] have chiefly addressed the selection of informative destinations, whereas the responsibilities that follow such a selection are often left unspecified. In practice, frontiers may become unreachable as the map evolves, navigation actions may fail or stall, and the disappearance of admissible goals must be distinguished from a merely temporary inability to reach them. These broader supervisory responsibilities are commonly left to application-specific orchestration logic, thereby limiting reuse across robotic platforms.

A further challenge arises when mapping is performed in dynamic environments populated by people, carts, and movable objects. Such observations are essential for collision avoidance, yet need not become part of the persistent environmental representation, since transient objects incorporated

into an occupancy map may leave residual structures that reduce consistency. Dynamic-scene mapping has been addressed through tightly coupled SLAM, semantic-perception, and learning-based methods [7], [8]; however, such approaches may require substantial computational resources or modification of the mapping system. To address these requirements, this paper presents a modular ROS 2 architecture integrating multimodal scan fusion, static–dynamic obstacle filtering, SLAM-based map construction, frontier-based exploration, and Nav2-based navigation.

The filtering module serves as a supporting preprocessing component that separates transient observations from persistent structure while preserving conventional scan-based interfaces; its internal method is not examined herein and lies beyond the scope of the present work. LiDAR scans and point-cloud observations are reduced to a unified planar representation using the multimodal `LaserScan` method of Noel *et al.* [9]. The five components are coordinated by a program manager that governs the mission from distributed launch through orderly termination. Rather than proposing a new SLAM, navigation, or scan-fusion algorithm, this work addresses the architectural and decision-making layers required to bind established components into a complete autonomous mapping system, implemented upon the CALL-M mobile manipulator [10] and evaluated in simulated and physical environments.

Accordingly, this paper presents two principal contributions. First, it demonstrates a complete ROS 2 software architecture for autonomous mapping, wherein perception, mapping, exploration, and navigation are united within a supervised mission workflow spanning initialization, map preservation, and coordinated termination. Second, it introduces a modular and reusable exploration-management framework that extends beyond frontier detection to orchestrate candidate evaluation, navigation-goal supervision, failure recovery, temporary exclusion of problematic regions, progress assessment, and mission completion. Requiring only an occupancy-grid map, robot-pose information through TF, and a Nav2-compatible navigation interface, the framework may be adopted by compatible ROS 2 mobile-robot stacks through external configuration, without modification of its underlying implementation.

The remainder of this paper is organized as follows: Section II reviews related work, Section III presents the experimental platform, Section IV details the methodology, Section V reports the experimental evaluation, and Section VI concludes the paper.

[†]CNRS-AIST JRL (Joint Robotics Laboratory), IRL, Tsukuba, Japan.
Emails: raymundo.gomez.rv@gmail.com, rafael.cisneros@aist.go.jp

*Corresponding author

II. RELATED WORK

A. Autonomous Exploration Algorithms

Autonomous exploration has long focused on selecting informative destinations from incomplete environmental representations. Frontier-based exploration, introduced by Yamauchi [3], identifies boundaries between known free space and unexplored regions and remains widely used for occupancy-grid maps. Subsequent approaches have incorporated expected information gain, frontier size, travel cost [4], randomized search structures [5], and information-theoretic criteria [6]. These principles are likewise embodied in ROS exploration packages such as `explore_lite` [11], which couples frontier selection to the navigation stack.

Goal selection alone, however, does not govern the complete mapping mission. Practical deployment further requires navigation-goal supervision, failure recovery, exclusion of problematic regions, progress assessment, and reliable termination, responsibilities that are commonly left to application-specific orchestration logic.

B. Mission Supervision and Recovery

Higher-level robotic coordination has traditionally been expressed through finite-state machines and, more recently, behavior trees, which provide a more modular and reactive means of composing supervisory behavior [12], [13]. Nav2 employs behavior trees to coordinate navigation execution and recovery actions.

Such mechanisms provide the means to express supervision, but do not themselves define how an autonomous mapping mission should manage failed goals, reconsider temporarily inaccessible regions, or distinguish completion from a recoverable impasse. The present work addresses this gap through a reusable exploration-management framework that unifies frontier evaluation, navigation-goal supervision, failure memory, recovery, and mission termination. Its distinction lies in treating these exploration-specific responsibilities as a reusable management layer, rather than as either a frontier-selection heuristic or platform-specific mission logic.

III. EXPERIMENTAL PLATFORM

The proposed autonomous-mapping system is implemented on CALL-M [10], a ROS 2-based mobile manipulator developed for navigation, perception, and interaction within structured indoor environments. As shown in Fig. 1, the robot combines a TriOrb omnidirectional base, whose holonomic motion suits narrow aisles and shelf-oriented environments, with a UR7e arm and a Robotiq 2F-85 gripper. Environmental perception is provided by two RPLIDAR A1M8 laser scanners, two side-facing ZED Mini stereo cameras, and a forward-facing Intel RealSense D435i mounted at the end effector, supplemented by four short-range ultrasonic sensors covering the front, rear, left, and right approaches. The cameras extend obstacle perception beyond the single horizontal scanning plane of the LiDARs.

Computation is distributed across three machines: an NVIDIA Jetson Orin Nano dedicated to the three camera streams, thereby reducing the volume of point-cloud data

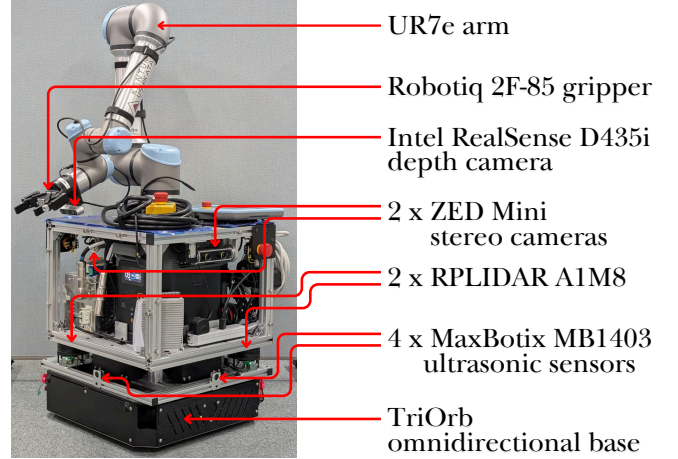


Fig. 1. CALL-M mobile-manipulation platform.

close to its source; an Intel NUC 13 connected to every remaining device and serving as the principal computing resource; and an external operator computer used for supervision and visualization. All three machines participate in a common ROS 2 network, allowing topics and services to be exchanged irrespective of the computer on which they originate.

IV. METHODOLOGY

The proposed autonomous-mapping system is organized as the closed-loop architecture shown in Fig. 2. Five principal processing components transform raw environmental observations into autonomous motion: (i) multimodal scan fusion; (ii) static–dynamic obstacle filtering; (iii) SLAM-based map construction; (iv) frontier-based exploration; and (v) Nav2-based navigation. Their mere coexistence, however, does not suffice to constitute an autonomous mapping mission. The components possess dependencies upon one another, become ready at different instants, execute across several computers on the physical platform, and may fail independently during operation. They are therefore placed under a program manager that governs initialization, verifies system readiness, admits autonomous operation only once the required interfaces are available, preserves the resulting map, and coordinates termination. This supervised autonomous-mapping architecture constitutes the first contribution of the present work.

Within this architecture, Node 4 is developed further as the second contribution: a modular and reusable exploration-management framework that extends beyond frontier selection to govern candidate evaluation, navigation-goal supervision, failure recovery, progress assessment, and mission completion. Its interfaces, frames, topics, thresholds, and operational parameters are declared externally, so that the framework may be transferred to compatible ROS 2 mobile-robot stacks without modifying its implementation. Node 2, by contrast, is employed here as a supporting preprocessing component for operation in dynamic environments; its internal filtering method is not treated as a contribution of the present work and is reserved for a separate study.

A. Supervised Autonomous-Mapping Software System

The processing chain proceeds from perception to motion while the map returned by that motion closes the exploration loop. Node 1 reduces the heterogeneous range observations to a common planar representation; Node 2 separates the observations admitted to the persistent map from transient ones; Node 3 constructs the occupancy grid; Node 4 examines that evolving map, elects the next destination, and thereafter supervises the resulting navigation goal through execution, recovery, and completion; and Node 5 executes the corresponding motion. As the robot advances, the sensors acquire the environment from a new vantage, the map grows, and Node 4 is furnished with the evidence upon which its next decision rests. Exploration is thus not a linear sequence of independent modules, but a closed computational cycle whose successive traversals enlarge the representation that determines the motion to follow.

1) *Program manager*: Above this processing cycle resides the program manager. Unlike the five nodes, it does not transform environmental data and is not itself part of the exploration policy. Its purpose is instead to establish and preserve the operational conditions under which the complete system may safely proceed.

Startup is deliberately ordered according to data dependency. The sensing pipeline is launched first, and execution advances only once the fused planar sensor representation required by the downstream mapping pipeline begins to publish. The static environmental scan is then made available to SLAM Toolbox, after which the manager awaits publication of the occupancy map. Navigation is admitted only after its action interface becomes available, whereupon the exploration-management framework is launched in its non-actuating WAITING state. Autonomous exploration is therefore never enabled as a side effect of process creation: a separate confirmation is required, and a START command is issued only after the complete chain has satisfied its readiness conditions. This ordering prevents downstream modules from acting upon absent or immature upstream interfaces.

The same sequence is preserved on the physical platform despite its distributed execution. Perception processes associated with the cameras execute on the Jetson, while the base interfaces, scan fusion, filtering, SLAM, and Nav2 are hosted principally on the NUC; the operator computer retains supervision and visualization. The program manager

abstracts this distribution by launching the appropriate local or remote processes while exposing the system as one ROS 2 graph. Consequently, the transition from simulation to hardware alters the placement and configuration of processes rather than the logical progression of the mapping mission.

2) *Node 1: Multimodal scan fusion*: Node 1 receives the two planar RPLIDAR scans together with the point clouds produced by the two ZED Mini cameras and the RealSense D435i, and reduces them to a common planar representation using the unified `LaserScan` method of Noel *et al.* [9]. The three-dimensional observations are not projected indiscriminately onto the ground plane; rather, the conversion takes the geometry of CALL-M into account and retains only those structures whose vertical extent could physically intersect the robot. During the present experiments, the UR7e remained in a fixed mapping configuration, yielding a maximum platform height of approximately $h_{\max} = 1.35$ m. Consequently, geometry lying entirely above this height is omitted from the planar obstacle representation, whereas any structure extending into the robot's occupied vertical envelope is preserved as an obstacle.

This distinction allows the projected scan to represent collision-relevant geometry rather than merely all observed geometry. For example, an overhead shelf or rack whose lowest surface lies above 1.35 m need not be marked as an obstacle, since CALL-M may pass beneath it, whereas a shelf, beam, or protrusion extending below this clearance is retained because it may intersect the platform. The resulting camera-derived scans therefore preserve the three-dimensional collision information relevant to CALL-M while expressing it through the two-dimensional `LaserScan` interface required by the downstream mapping and navigation components.

The three camera-derived scans are then merged angularly with the two RPLIDAR scans to form the unified observation `/unified_scan`. This common representation provides the downstream components with a single planar sensing interface while retaining obstacle information gathered from both the planar LiDARs and the depth cameras.

3) *Node 2: Static-dynamic obstacle filtering*: Node 2 receives `/unified_scan` and separates persistent environmental structure from transient observations, publishing `/static_scan` for SLAM-based map construction and `/dynamic_scan` for local obstacle avoidance and dynamic-object tracking. In this manner, moving objects remain available to the navigation subsystem for immediate collision avoidance while being prevented from accumulating as enduring occupancy within the persistent map.

The filtering algorithm itself is not examined herein; Node 2 is retained in the present architecture as the pre-processing component that enables the complete mapping system to operate within dynamic indoor environments.

4) *Node 3: SLAM-based map construction*: Node 3 supplies `/static_scan` to SLAM Toolbox, which constructs the occupancy-grid map used throughout the remainder of the system. SLAM Toolbox also furnishes the `map→odom` correction from which the robot pose in the global map

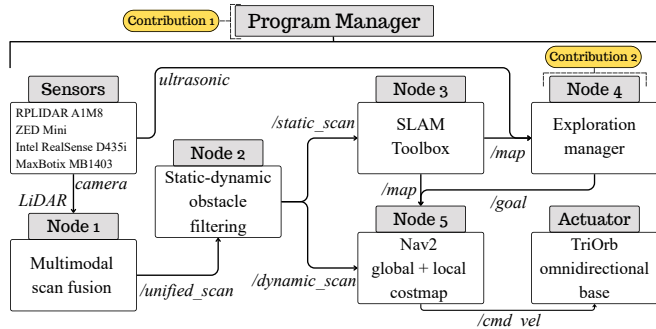


Fig. 2. Software architecture of the proposed autonomous-mapping system.

frame is obtained. The evolving occupancy grid consequently serves two purposes: it is the persistent product of the mapping mission and, simultaneously, the representation upon which the exploration manager reasons about what remains unknown.

5) *Node 4: Frontier-based exploration:* Node 4 receives the occupancy map together with the robot pose and constitutes the decision-making element of the mapping cycle. It identifies the boundaries between observed free space and unexplored regions, constructs candidate destinations from those boundaries, elects the candidate considered most useful, and dispatches it through the navigation interface. Its responsibility does not cease upon dispatch: the node also supervises the ensuing goal, records failures, governs temporary exclusions, requests subsequent decisions, and determines when exploration has reached completion. These mechanisms form the exploration-management framework developed in Section IV-B.

6) *Node 5: Nav2-based navigation:* Node 5 executes the destination elected by Node 4 using Nav2. The occupancy map provides the persistent environmental representation used for global planning, while `/unified_scan` is supplied to the global costmap and `/dynamic_scan` to the local costmap. In this manner, the global navigation layer retains the complete fused range representation of the surroundings, whereas the local layer is furnished specifically with the transient obstacles that require immediate avoidance. Nav2 then computes and executes the motion required to approach the selected frontier and publishes the velocity commands that drive the TriOrb omnidirectional base.

The distinction between persistent mapping and immediate obstacle avoidance is deliberate. A transient observation need not become part of the lasting map in order to influence the robot's present motion: the mapping branch receives the filtered static representation, while the navigation subsystem retains both the fused environmental observation and the transient-obstacle stream required for safe execution. Once Node 5 moves the base, a new portion of the environment enters view and the cycle begins anew.

B. Exploration-Management Framework

The exploration-management framework constitutes the decision-making and supervisory layer of the autonomous-mapping system. Rather than limiting exploration to the repeated selection of frontier goals, it governs the complete process by which candidate regions are generated, evaluated, dispatched, supervised during navigation, reconsidered after failure, and eventually judged exhausted or complete. Its behavior is summarized in Fig. 3.

The framework is designed independently of CALL-M-specific sensing hardware. It requires only an occupancy-grid map, the robot pose through TF, and a Nav2-compatible navigation interface. Topics, reference frames, and operational parameters are declared externally, so that the same implementation may be transferred to a compatible ROS 2 mobile-robot stack without modification of its underlying exploration logic.

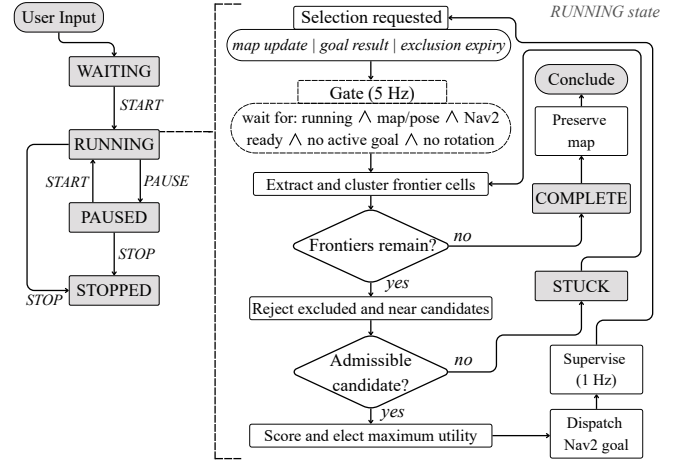


Fig. 3. Exploration-management framework overview.

Algorithm 1 summarizes the operative cycle of the framework, wherein candidate selection passes through a single supervisory gate until exploration is judged complete.

1) *Mission states and control:* The framework operates through four principal control states: WAITING, RUNNING, PAUSED, and STOPPED. It begins in WAITING, wherein map and pose information may already be received but no navigation goal may be issued. Following START, the framework enters RUNNING and the autonomous exploration cycle is enabled.

A PAUSE command interrupts the active exploration transaction while preserving the accumulated map, navigation history, and failure information. A subsequent START therefore resumes rather than reinitializes the mission. STOP, by contrast, is terminal: active motion is canceled and the current instance may not be restarted without relaunching the framework. The additional SAVE_MAP command allows the current occupancy map to be preserved without otherwise altering the exploration state.

This separation between process execution and permission to move permits the exploration manager to be launched during system initialization while remaining non-actuating. Autonomous operation begins only after the required map, robot pose, and navigation interfaces have become available and the program manager has explicitly issued START.

2) *Frontier generation and candidate validation:* The framework identifies frontier cells directly from the occupancy grid, where free, occupied, and unknown cells are represented by 0, 100, and -1, respectively. A free cell is considered a frontier when at least one of its four immediate cardinal neighbors remains unknown. Restricting detection to cardinal adjacency avoids creating spurious frontiers where unknown space touches free space only diagonally across an obstacle corner.

The detected frontier cells are then grouped into eight-connected clusters through breadth-first traversal, and clusters smaller than a configured threshold are discarded as map noise. The centroid of each remaining cluster is used only to indicate the general region toward which exploration should proceed; it is not sent directly to Nav2. Since a centroid may

Algorithm 1 Exploration-management framework

```
1: Initialize state  $\leftarrow$  WAITING
2: Initialize recent-goal and failure histories
3: while state  $\neq$  STOPPED do
4:   Receive map, robot pose, commands, and navigation
     events
5:   if START received and required interfaces are ready
     then
6:     state  $\leftarrow$  RUNNING
7:   end if
8:   if PAUSE received then
9:     Cancel active goal
10:    state  $\leftarrow$  PAUSED
11:  end if
12:  if STOP received then
13:    Cancel active goal and preserve map
14:    state  $\leftarrow$  STOPPED
15:  end if
16:  if state = RUNNING and selection is requested then
17:    if no navigation goal is active then
18:      Extract and cluster frontier cells
19:      Generate and validate candidate goals
20:      Remove temporarily excluded candidates
21:      if admissible candidates exist then
22:        for each candidate  $i$  do
23:          Compute utility  $J_i$ 
24:        end for
25:         $g^* \leftarrow \arg \max_i J_i$ 
26:        Dispatch  $g^*$  through NavigateToPose
27:      else
28:        Evaluate mission-completion criteria
29:        if completion criteria are satisfied then
30:          Preserve map and report COMPLETE
31:        end if
32:      end if
33:    end if
34:  end if
35:  if active goal succeeds or near-goal distance is reached
     then
36:    Update recent-goal history
37:    Clear corresponding failure history
38:    Request a new selection
39:  else if goal is rejected, aborted, or timed out then
40:    Update spatial failure history
41:    Apply temporary or terminal exclusion
42:    Request a new selection
43:  end if
44: end while
```

lie in unknown or occupied space, the framework instead selects an actual free frontier cell near the centroid and verifies that it constitutes a suitable navigation target.

A candidate is rejected when it lies too close to the boundary of the expanding occupancy grid, lacks sufficient clearance from known obstacles, or coincides with a high-cost cell in the global costmap. Unknown costmap cells are not rejected merely for being unknown, since frontier goals

naturally lie beside unexplored space. Candidates already very near the robot are likewise omitted, because navigating toward them is unlikely to reveal a meaningful amount of new environment.

3) *Candidate utility*: For each remaining frontier cluster i , a base utility is computed from its size and distance from the robot:

$$J_i^0 = \frac{N_i}{\max(d_i^2, \epsilon)}, \quad (1)$$

where N_i denotes the number of cells belonging to the frontier cluster, d_i is the Euclidean distance from the robot to its selected goal cell, and ϵ prevents an excessively large score at very short distance. This form favors frontiers that expose larger unexplored regions while avoiding unnecessarily long motions toward small or distant candidates.

Frontier size and distance alone, however, may still produce inefficient behavior. In particular, the robot may repeatedly alternate between attractive frontiers located on opposite sides of the mapped area, yielding a back-and-forth trajectory that increases travel distance and interrupts the natural progression of exploration. To encourage a smoother and more coherent motion through the environment, the framework retains a preferred exploration direction derived from recent successful travel. If θ_i denotes the angular difference between that direction and the direction toward candidate i , directional continuity is introduced through

$$C_i = 1 + w_c \cos(\theta_i), \quad (2)$$

where w_c determines the strength of the preference. The present implementation uses $w_c = 0.45$. Candidates lying broadly along the recent direction of travel are therefore favored, while candidates requiring a strong reversal receive a lower score. The preferred direction is updated only after a sufficiently long successful traversal and retains part of its previous value, preventing short motions from causing abrupt changes in the overall exploration flow.

A second history term discourages immediate return to regions that have only recently been reached. Let R_i denote the corresponding recent-goal factor. Candidates lying within a configured radius of the recent-goal history receive a penalty $R_i < 1$, whereas all others retain $R_i = 1$. This reduces short-range oscillation between neighboring frontier regions and encourages the robot to continue toward portions of the map that are more likely to provide new information. The final candidate score is therefore

$$J_i = J_i^0 C_i R_i. \quad (3)$$

Both history terms remain soft preferences rather than hard constraints. A candidate opposing the recent direction of travel may still be selected when its frontier size and distance make it clearly more valuable. In this manner, the utility function seeks a practical balance between information gain, travel cost, and trajectory continuity, producing exploration paths that are less prone to repeated reversals and unnecessary revisiting of already explored regions.

4) *Event-driven goal supervision*: Selection is event-driven but deliberately separated from the ROS 2 callbacks that generate those events. A newly received occupancy map, a completed navigation action, or the expiration of a temporary exclusion may each indicate that another frontier ought to be considered. Acting immediately inside those callbacks, however, could allow several nearly simultaneous events to initiate competing goal selections.

The callbacks therefore request selection without performing it directly. A single supervisory gate, evaluated at 5 Hz, executes the pending request only when the framework remains in `RUNNING`, the required environmental state is available, the navigation action server is ready, and no other goal transaction is active. Every candidate election consequently passes through the same decision point, ensuring that at most one frontier is being acted upon at any instant.

Once a candidate has been selected, its pose is published for monitoring and dispatched through `NavigateToPose`. The framework then supervises the goal throughout its execution rather than immediately returning to candidate selection. If the robot approaches within 0.55 m of the frontier, the goal is considered reached and exploration proceeds to the next decision. Otherwise, the navigation action remains active until success, failure, rejection, or the configured timeout. The timeout is measured from the actual goal-dispatch instant, providing a consistent bound upon the time devoted to any single navigation attempt.

5) *Failure memory and recovery*: The framework supervises the complete life cycle of every navigation goal, including transmission, acceptance, execution, cancellation, and final result. A rejected, aborted, or timed-out goal is treated as evidence that the corresponding region is presently difficult to reach.

Failure history is stored spatially rather than by exact occupancy-grid cell. This distinction is important because the apparent position of a frontier may shift slightly as SLAM expands or revises the map. A newly generated candidate lying sufficiently close to an earlier failed candidate therefore inherits its failure history.

The retry policy becomes progressively more restrictive with repeated failure. A first failure excludes the corresponding region for 30 s, while a second extends the exclusion to 120 s. Following a third failure, the region is considered exhausted for the remainder of the mission. Once a temporary exclusion expires, the region becomes admissible again and a new selection is requested. Successful navigation to the same region clears its accumulated failure history.

This policy prevents immediate repetition of a navigation action that has already proved unsuccessful, while avoiding the opposite extreme of permanently discarding a region after a single transient obstruction. It therefore permits the framework to reconsider a frontier after the environment or navigation conditions have changed.

6) *Illustrative exploration example*: Figure 4 illustrates a simplified decision made by the exploration manager. The cyan marker denotes the robot, while F_1 – F_4 represent frontier candidates extracted from the boundary between

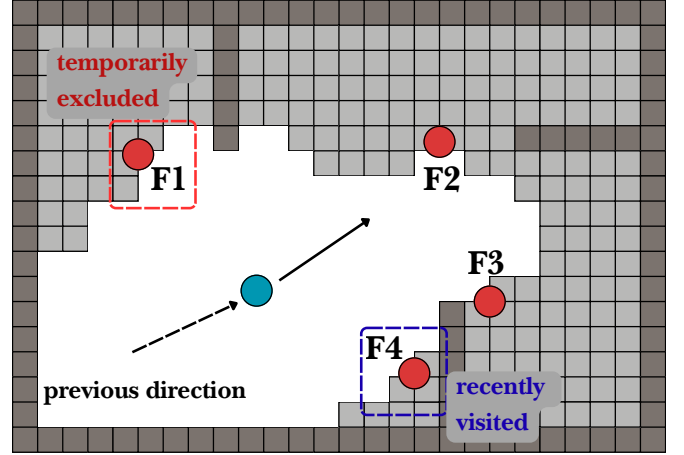


Fig. 4. Toy example of frontier evaluation and supervisory memory.

TABLE I
ILLUSTRATIVE FRONTIER EVALUATION.

Frontier	N_i	d_i (m)	C_i	R_i	J_i
F_1	22	1.8	0.65	1.00	Excluded
F_2	30	3.0	1.30	1.00	4.33
F_3	18	2.4	0.85	1.00	2.66
F_4	20	2.0	0.90	0.40	1.80

known free space and the surrounding unknown region. The dashed arrow records the preceding direction of travel, from which the directional-continuity term is derived. The solid arrow indicates the resulting preferred progression of the exploration process.

The candidates are not considered on distance alone. Frontier F_1 lies close to the robot, but occupies a region temporarily excluded after an earlier navigation failure and is therefore removed before utility evaluation. Frontiers F_2 and F_3 remain fully admissible, whereas F_4 lies near a recently visited region and consequently receives the history penalty $R_i < 1$. Thus, although F_4 remains a valid frontier, its recent traversal makes an immediate return less desirable.

Table I gives one numerical realization of the decision. For clarity, the values are illustrative rather than measurements from the experiments. Since F_1 is temporarily excluded, no utility is evaluated for it. Among the remaining candidates, F_2 obtains the largest score because its frontier size and favorable directional continuity outweigh its greater distance. Although F_4 is comparatively near, its recent-goal penalty lowers its utility. The example therefore shows how geometric desirability, exploration history, and failure memory act together before a navigation goal is dispatched.

7) *Mission completion*: The disappearance of selectable frontiers at one instant is not by itself sufficient evidence that exploration has finished. An evolving SLAM map may temporarily contain no admissible target while new observations are still being incorporated or previously failed regions remain under cooldown. Completion is therefore assessed only across successive occupancy-grid updates.

The first completion criterion requires that no eligible frontier remain for three consecutive new-map refreshes. A temporarily excluded frontier does not satisfy this condition,

because the framework already intends to reconsider it once its cooldown expires.

A second criterion addresses cases in which frontier structures remain visible but continued attempts no longer enlarge the map. Consecutive occupancy grids are compared after accounting for changes in map extent, and the percentage of known cells whose occupancy state has changed is calculated. If this change remains below 0.1 % for five consecutive map refreshes, and every remaining meaningful frontier has already failed at least twice, further exploration is considered unlikely to yield useful progress.

Thus, mission completion arises from repeated evidence of either frontier exhaustion or map convergence rather than from a single unsuccessful search. Once either condition is satisfied, further candidate selection is suppressed, the framework reports COMPLETE, and map preservation is requested.

V. EXPERIMENTAL EVALUATION

The proposed system is evaluated in simulation and subsequently deployed on the physical CALL-M platform. The simulation study examines whether the supervised architecture and exploration-management framework improve mission reliability relative to a conventional nearest-frontier strategy. Because simulation provides repeatable environmental conditions and known geometry, it is used for the quantitative comparison. The hardware study then verifies that the complete architecture transfers to the distributed physical platform and can autonomously carry a mapping mission to completion. The accompanying video illustrates the operation of the complete system in both settings.

Fig. 5 presents the simulated environment and a representative map produced by the complete system. Fig. 5(a) shows the $24\text{ m} \times 24\text{ m}$ warehouse, containing workbenches, crates, and shelving racks of differing clearance. The green racks lie above the collision envelope defined for CALL-M and may therefore be traversed beneath, whereas the red racks obstruct the platform. Four pedestrian actors circulate continuously along the paths indicated by the red arrows, repeatedly crossing the robot's route during exploration. Fig. 5(b) shows the map obtained after one fully autonomous mission.

The resulting occupancy grid reflects the traversability represented by the multimodal sensing pipeline: obstructing racks remain in the map, whereas overhead structures beneath which CALL-M may safely pass are omitted. Transient pedestrian observations likewise do not remain as persistent occupancy. This example illustrates the output of the complete architecture; quantitative evaluation of the proposed supervisory system is given below.

A. Experiment 1: Mission Completion and Recovery

Two conditions are compared: the full system and a minimal-supervision baseline. Both share the same frontier detector, candidate-safety checks, SLAM front-end, and Nav2 NavigateToPose interface, while differing in goal selection and supervisory management. The baseline follows a conventional nearest-frontier strategy derived from

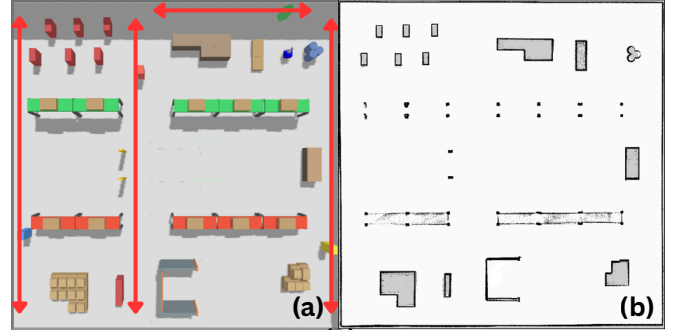


Fig. 5. (a) Warehouse with pedestrian paths (red). (b) Final map.

Yamauchi [3]: the nearest valid frontier is dispatched directly to Nav2, and another frontier is selected when the navigation action concludes. It contains no supervisory goal timeout, spatial failure memory, escalating retry policy, history-dependent candidate scoring, or convergence-based termination. The full framework instead evaluates frontier size, distance, directional continuity, and recent-goal history while supervising each navigation attempt through timeout handling, failure memory, retry cooldowns, and mission-completion logic.

The experiments are conducted in the environment of Fig. 5(a), with each condition repeated ten times and object placement held identical throughout. A run counts as completed if the manager terminates without intervention before the 3000 s mission timeout, and coverage is the mapped known area as a fraction of the 566.4 m^2 of reachable free space.

TABLE II
EXPERIMENT 1 — EXPLORATION PERFORMANCE ($n = 10$).

Metric	Full	Baseline
Missions completed	10 / 10	7 / 10
Coverage of reachable area (%)	95.3 ± 2.4	92.5 ± 11.0
Exploration time (sim s)	1383 ± 274	1402 ± 653
Goals issued	56.1 ± 5.5	161.9 ± 18.1
Distance per goal (m)	3.86 ± 0.53	1.21 ± 0.18
Frontier evaluations	67.6 ± 13.1	174.3 ± 13.3
Frontier computation (ms)	15.1 ± 1.3	19.7 ± 9.9

Table II shows that the principal effect of supervision is mission reliability rather than raw exploration speed. Mean completion time is similar between the two conditions, yet all ten full-system runs terminated autonomously, compared with seven of ten baseline runs. Without a supervisory timeout, a navigation goal that remains unresolved can arrest the exploration loop; without failure memory, a repeatedly unreachable region may subsequently be selected again. The most severe baseline run terminated at only 61.5 % coverage after 58 failed goals. Accordingly, coverage variability fell from 11.0 % to 2.4 %, while the difference in mean coverage remained below three percentage points. The framework therefore contributes chiefly by removing catastrophic or non-convergent runs rather than by increasing the coverage attained by an already successful mission.

This distinction is central to unattended deployment: a

mapping system that achieves high coverage only when navigation proceeds favorably is less useful than one able to detect stalled progress, recover from failed regions, and terminate reliably without operator intervention.

The remaining metrics characterize the different exploration behavior produced by the complete framework. The full system issued only 56.1 ± 5.5 goals, compared with 161.9 ± 18.1 for the baseline, while the mean distance traveled per issued goal increased from 1.21 m to 3.86 m. The baseline consequently devoted many more decisions to short movements between nearby frontiers, whereas the full framework sustained longer periods of progress before requiring another goal. This behavior is consistent with the directional-continuity and recent-goal terms introduced in Section IV-B, although their individual effects are not separated in the present experiment.

Fewer goal decisions also reduced the number of frontier evaluations from 174.3 ± 13.3 to 67.6 ± 13.1 per mission. Frontier computation itself remained inexpensive in both conditions, requiring 15.1 ± 1.3 ms for the full framework and 19.7 ± 9.9 ms for the baseline. The principal benefit is therefore not faster individual frontier computation, but a reduction in how often the exploration process must reconsider its next destination.

B. Hardware Deployment on CALL-M

The complete architecture was subsequently deployed on CALL-M in a $6.80 \text{ m} \times 4.90 \text{ m}$ meeting room. All processing components and the program manager were active, with computation distributed across the onboard NUC, Jetson, and operator computer as described in Section III. Five autonomous mapping sessions were conducted from the same initial region while personnel moved through the workspace. Each session was allowed to proceed from initialization through autonomous exploration and map preservation without intervention in the exploration decision process. Room dimensions were measured manually to provide a simple geometric reference for the resulting maps.

TABLE III
HARDWARE DEPLOYMENT ON CALL-M ($n = 5$).

Metric	Mapped	Measured	Error
Short wall separation (m)	4.85 ± 0.05	4.90	-0.05
Long wall separation (m)	7.13 ± 0.07	6.80	+0.33
<i>Session statistics</i>			
Sessions completed	5 / 5 (100 %)		
Mapped area (m^2)	43.0-57.1		

All five hardware sessions reached autonomous completion, demonstrating that the same supervisory workflow used in simulation transfers to the distributed physical platform. The resulting maps were also geometrically repeatable. The short wall separation was reconstructed as 4.85 ± 0.05 m relative to the measured 4.90 m, while the long separation was 7.13 ± 0.07 m relative to 6.80 m. The corresponding absolute errors are 0.05 m (1.0 %) and 0.33 m (4.9 %). Run-to-run variation remained considerably smaller than the long-axis bias, suggesting a systematic geometric error rather

than poor repeatability across missions. The hardware results therefore verify not only map repeatability, but also that the complete supervisory workflow remains functional under distributed real-time execution.

VI. CONCLUSION

This paper presented a modular ROS 2 architecture for autonomous mapping in unknown and dynamic indoor environments, together with a reusable exploration-management framework for supervision beyond frontier selection. The system coordinates perception, mapping, exploration, and navigation from initialization through mission completion, while combining candidate evaluation, navigation-goal supervision, failure memory, recovery, and convergence-based termination. In simulation, the framework completed all ten missions, compared with seven of ten for the nearest-frontier baseline, while reducing the mean number of issued goals from 161.9 to 56.1. All five hardware sessions on CALL-M likewise reached autonomous completion, demonstrating that the same supervisory principles remain effective under distributed physical execution.

The evaluation remains limited to one simulated environment and five hardware sessions, and portability has not yet been demonstrated on a second robotic platform. Future work will therefore examine additional environments and robots, together with controlled ablations of the candidate-history, failure-recovery, and mission-completion mechanisms. The results nevertheless indicate that reliable autonomous mapping depends not only on capable mapping and navigation, but also on a supervisory layer able to bind them into a coherent mission, sustain useful progress in the presence of failures, and determine when that mission has truly reached completion.

REFERENCES

- [1] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *J. Open Source Softw.*, 2021.
- [2] S. Macenski *et al.*, "The Marathon 2: A navigation system," in *IROS*, 2020.
- [3] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *CIRA*, 1997.
- [4] H. H. González-Baños and J.-C. Latombe, "Navigation strategies for exploring indoor environments," *Int. J. Robot. Res.*, 2002.
- [5] H. Umari and S. Mukhopadhyay, "Autonomous robotic exploration based on multiple rapidly-exploring randomized trees," in *IROS*, 2017.
- [6] F. Bourgault *et al.*, "Information based adaptive robotic exploration," in *IROS*, 2002.
- [7] B. Bescos *et al.*, "DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes," *IEEE Robot. Autom. Lett.*, 2018.
- [8] C. Yu *et al.*, "DS-SLAM: A semantic visual SLAM towards dynamic environments," in *IROS*, 2018.
- [9] O. Noel *et al.*, "A lightweight approach to efficient multimodal 2D navigation and mapping: Unified Laser-Scans as an alternative to 3D methods," in *SII*, 2025.
- [10] D. Vázquez *et al.*, "Development of a semi-autonomous manipulation pipeline for robotic shelf-picking operations," in *SII*, 2026.
- [11] J. Horner, "explore_lite: Lightweight frontier-based exploration," GitHub repository. [Online]. Available: <https://github.com/hrnr/m-explore>
- [12] A. Marzinotto, M. Colledanchise, C. Smith, and P. Ögren, "Towards a unified behavior trees framework for robot control," in *ICRA*, 2014.
- [13] M. Colledanchise and P. Ögren, "How behavior trees modularize hybrid control systems and generalize sequential behavior compositions, the subsumption architecture, and decision trees," *IEEE Trans. Robot.*, 2017.